OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

NAME: T MOHAMMED SAIF

SRN: PES1UG24CS636

CLASS: 4K

Phase 1: Object Storage Foundation

Screenshot 1A: Output of `./test_objects` showing all tests passing.

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

Screenshot 1B: `find .pes/objects -type f` showing the sharded directory structure.

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A: Output of `./test_tree` showing all tests passing.

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
saif@DESKTOP-R5O1597:~/PES1UG24CS630-pes-vcs$
```

Phase 3: The Index (Staging Area)

3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` — show the output.

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ make pes
make: 'pes' is up to date.
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ulimit -s unlimited
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ./pes init

echo "Hello PES" > file1.txt

./pes add file1.txt

./pes status

cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  tree.h
  untracked:  test_tree
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  Makefile
  untracked:  index.h
  untracked:  index.c
  untracked:  object.c
  untracked:  README.md
  untracked:  pes.c
  untracked:  tree.c

100644 f4e3b312fbd42e3d1a28a6193e4724ff95a376e21b7c845cd52e741362c7cc78 1776322475 10 file1.txt
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ make pes
make: 'pes' is up to date.
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ulimit -s unlimited
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ./pes init

echo "Hello PES" > file1.txt

./pes add file1.txt

./pes status

cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  tree.h
  untracked:  test_tree
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  Makefile
  untracked:  index.h
  untracked:  index.c
  untracked:  object.c
  untracked:  README.md
  untracked:  pes.c
  untracked:  tree.c

100644 f4e3b312fbd42e3d1a28a6193e4724ff95a376e21b7c845cd52e741362c7cc78 1776322475 10 file1.txt
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

Phase 4: Commits and History

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ ./pes log
commit 1edd3085fcc825c9bad2a180327bf39d6c3f4243b28e1b58ce2f76217bddc95a
Author: T Mohammed Saif <PES1UG24CS636>
Date:   1776323655

    Add farewellr auth!

commit 2014c3b3e8df81c6b42b1c0aceeddfab0746b1d3043ea72225591f639987dd07
Author: T Mohammed Saif <PES1UG24CS636>
Date:   1776323655

    Add worldellr auth!

commit 9dad748e66533477a0dbfcf904cb9c08c9c78af95cc4c302e6cdf6ffcb889df5
Author: T Mohammed Saif <PES1UG24CS636>
Date:   1776323655

    Initial committial commitEA

saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

Screenshot 4B: `find .pes -type f | sort` showing object store growth after three commits.

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/06/02a917c76aa78ba217010cc3f0d7a19073a5e9ec5950bcb33cd3497924f869
.pes/objects/1e/dd3085fcc825c9bad2a180327bf39d6c3f4243b28e1b58ce2f76217bddc95a
.pes/objects/20/14c3b3e8df81c6b42b1c0aceeddfab0746b1d3043ea72225591f639987dd07
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/2f/2d2118baae02e2cf993c0f3ff490a019cff5097b939e9fb11715be93a56f40
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8a/bd1c558f75d747bb8e45f3da31f5c4bb6515b1c8fd4ee6503feff6a17ae159
.pes/objects/9d/ad748e66533477a0dbfcf904cb9c08c9c78af95cc4c302e6cdf6ffcb889df5
.pes/objects/9e/524e3f0263db7bdf6756dad89728dd6f40a498dc9ec74b80e724b7e71680b0
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f4/e3b312fbd42e3d1a28a6193e4724ff95a376e21b7c845cd52e741362c7cc78
.pes/refs/heads/main
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$
```

Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

```
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ cat .pes/refs/heads/main
1edd3085fcc825c9bad2a180327bf39d6c3f4243b28e1b58ce2f76217bddc95a
saif@DESKTOP-R5OI597:~/PES1UG24CS636-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5: Branching and Checkout

Q5.1: Implementation of `pes checkout <branch>`

- **Files to change in `.pes/`:** 1. The `.pes/HEAD` file must be updated. Instead of containing a raw commit hash, it should be updated to a symbolic reference (e.g., `ref: refs/heads/<branch>`). 2. The **Index** must be overwritten. The VCS must load the root tree of the target branch and populate the index with those entries so the staging area matches the branch state.
- **Working Directory changes:** The VCS must physically synchronize the folder. It must delete files that don't exist in the target branch, update files that have changed, and create files that are new to the target branch.
- **Complexity:** This is complex because it is a **destructive operation**. If a crash occurs mid-checkout, the working directory is left in a "hybrid" state. Implementing an atomic "rollback" mechanism to restore the original state upon failure is the primary challenge.

Q5.2: Detecting a "Dirty Working Directory" Conflict

To detect this using the index and object store, you perform a **three-way comparison**:

1. **Compare Working Directory to Index:** Check the `mtime` and `size` of the file on disk against the Index. If they differ, the file has uncommitted changes (it is "dirty").
2. **Compare Index to Target Branch:** Check if the hash of the file in the current Index matches the hash of the same file in the Target Branch's tree.
3. **The Conflict Rule:** If the file is modified in the working directory **AND** the version in the target branch is different from the version in your current index, `checkout` must refuse to proceed. This prevents the VCS from overwriting the user's unsaved work.

Q5.3: "Detached HEAD" State

- **What happens:** You can still make commits. `HEAD` will simply update to the new commit's hash. However, since no branch (like `main`) points to these commits, they are "anonymous."
- **Recovery:** If you switch to another branch, the detached commits become unreachable. A user can recover them by:
    1. Looking at the **Reflog** (if implemented) to find the lost hashes.
    2. Manually creating a branch at that hash: `pes branch <name> <hash>`.

OPTERATING SYSTEM
ORANGE 2: PES –VCS LAB REPORT

Phase 6: Garbage Collection (GC)

Q6.1: Mark-and-Sweep Algorithm

- **The Algorithm:**
  1. **Mark:** Start at all "known truths" (all branch heads in `refs/heads/`, `HEAD`, and tags). Recursively traverse every commit, its tree, and all sub-blobs.
  2. **Sweep:** Scan the `.pes/objects` directory. Any file (hash) not encountered during the "Mark" phase is unreachable and can be safely deleted.
- **Data Structure:** A **Hash Set** or **Bloom Filter** is used to store "reachable" hashes. This ensures $O(1)$ lookup and prevents redundant processing of shared objects (deduplication).
- **Estimation:** You visit all 100,000 commits. While there are 50 branches, many share history. You visit each unique commit, tree, and blob once. In a typical repo, this could total 500,000 to 1,000,000 objects visited.

Q6.2: GC Race Conditions

- **The Race Condition:** Imagine User A is running `pes add`. The tool writes a new blob to the object store. Simultaneously, User B runs `pes gc`. Since the blob is written but the commit isn't finished yet, the GC sees the blob as "unreachable" and deletes it. User A then finishes the commit, but it points to a deleted object.
- **Git's Solution:** Git uses a **grace period** (pruning limit). By default, Git's GC will not delete any unreachable object that was created within the last 2 weeks (based on the file's `mtime`). This ensures that "in-flight" operations have plenty of time to finish and link their objects to a branch.